



TRAINING PROGRAMME

System Design Fundamentals

Design scalable, reliable systems and explain the trade-offs

21 HOURS

FOR Developers and engineers who design or review software systems



Mohamed Aziz BEN HAHHA

mohamedazizbenhaha.netlify.app

Overview

This workshop teaches participants to design software systems that scale and stay reliable, and to explain their choices. They work through the fundamentals of scalability, latency and consistency, API styles and authentication, caching and CDNs, data storage, replication and sharding, asynchronous messaging, and architecture styles with the patterns that keep them resilient and observable. Classic case studies put the pieces together, and the workshop closes with a final project in which each team designs a system for a brief and defends it in a peer review.

At a glance

#	PHASE	FOCUS	HOURS
01	Foundations, APIs & Caching	Scalability, CAP and consistency, API styles, authentication, caching, CDNs	6 h
02	Data, Messaging & Architecture	SQL vs NoSQL, replication, sharding, queues and Kafka, architecture styles, resilience	6 h
03	Case Studies	URL shortener, chat application, payment system, news feed	3 h
04	Final Project	A system designed for a brief, defended in peer review	3 h
	Buffer	Catch-up, revision, extra lab time	3 h
Total			21 h

Key facts

AUDIENCE	Developers and engineers who design or review software systems
PREREQUISITES	Programming in any language (course: Python Programming Fundamentals), or equivalent, and basic knowledge of databases and HTTP
DURATION	21 h: 18 h of chapters + 3 h buffer
FORMAT	Workshop: short lectures, each followed by design exercises in small groups; case studies and peer review
LAB ENVIRONMENT	One laptop per participant with a browser and a diagramming tool (Excalidraw or draw.io); small demonstrations with Redis and Kafka in Docker run by the trainer; no cloud account or cost
ASSESSMENT	Design exercises reviewed in each phase and a final design defended in a peer review; certificate of completion for participants who complete both
OUTCOME	Participants can turn requirements into a system design with APIs, data model, caching, messaging and a scaling plan, and argue its trade-offs

PHASE 01

6 H

Foundations, APIs & Caching

Participants learn the vocabulary of system design: scalability, latency and throughput, the CAP theorem and consistency models. They then choose how services communicate and authenticate, and where caching and CDNs cut latency and load. Validated exercise: a back-of-the-envelope capacity estimate and an API with a caching strategy for a photo-sharing service.

#	CHAPTER	CONTENT	FORMAT	HOURS
1.1	Fundamentals	Scalability, latency vs throughput, CAP theorem, consistency models, capacity estimates	Lecture + Workshop	3 h
1.2	APIs & Communication	REST, GraphQL, gRPC, webhooks, WebSockets, API gateways, authentication (OAuth 2.0, OpenID Connect)	Lecture + Workshop	1.5 h
1.3	Caching & CDN	Caching strategies, eviction, Redis, CDNs	Lecture + Workshop	1.5 h

PHASE 02

6 H

Data, Messaging & Architecture

Participants choose and scale data stores with replication, sharding and indexes, decouple services with queues, publish/subscribe and Kafka, and compare architecture styles with the patterns that keep them reliable: rate limiting, retries, circuit breakers, observability and service level objectives. Validated exercise: the data model, sharding key and order-processing flow of an online shop, with its failure handling.

#	CHAPTER	CONTENT	FORMAT	HOURS
2.1	Data	SQL vs NoSQL, replication, sharding, indexing	Lecture + Workshop	3 h
2.2	Asynchronous Systems	Queues, publish/subscribe, Kafka, idempotency, sagas	Lecture + Workshop	1.5 h
2.3	Architecture Styles	Monolith, microservices, event-driven, serverless; rate limiting, retries, circuit breakers, observability, SLOs	Lecture + Workshop	1.5 h

PHASE 03

3 H

Case Studies

Participants apply the whole method to classic problems, from requirements and estimates to the final architecture and its weak points, comparing their designs with each other. Validated exercise: one case study designed by each group and critiqued by another.

#	CHAPTER	CONTENT	FORMAT	HOURS
3.1	Case Studies	URL shortener, chat application, payment system, news feed	Lecture + Workshop	3 h

PHASE 04

3 H

Final Project

Teams of two or three design a system for a written brief, such as a ride-hailing application, reusing the methods of the workshop, then defend it in front of another team.

#	CHAPTER	CONTENT	FORMAT	HOURS
4.1	Brief & Design	Requirements and estimates, architecture diagram, data model, scaling plan	Project	1.5 h
4.2	Peer Review	Present the design and its trade-offs, review another team's design; feedback	Project	1.5 h

FINAL PROJECT

System Design for a Brief

A system designed from a written brief, such as a ride-hailing application: requirements and capacity estimates, architecture diagram, APIs, data model, caching and messaging, a scaling plan and the trade-offs behind each choice.

STACK Architecture diagrams (Excalidraw or draw.io), capacity estimates, design document

DELIVERABLE Design document with diagrams, data model and scaling plan, peer-review notes